Anti-Bluff Mandate Reinforcement — Group A-prime (closes §6.N-debt) Design

Date: 2026-05-05 (evening brainstorm; execution likely lands 2026-05-05 evening or 2026-05-06) **Spec ID:** Group A-prime — successor to Group A (`docs/superpowers/specs/2026-05-05-anti-bluff-mandate-reinforcement-design.md`) **Status:** Approved (user said "all ok" to architecture, components, data flow, error handling, testing, order-of-ops; "all ok" again on Q8) **Implementation skill:** `writing-plans` (next)

Forensic anchor

Group A landed §6.N (Bluff-Hunt Cadence Tightening + Production Code Coverage) as constitutional doc-only updates and explicitly deferred mechanical enforcement via a §6.N-debt placeholder. The placeholder text (in root `CLAUDE.md`) reads:

> "The pre-push hook enforcement clauses (6.N.1.2 + 6.N.1.3 above) are documented but NOT yet implemented. Implementation is deferred to the Group A-prime spec, which the parent Group A spec spawns as the next brainstorming + writing-plans cycle."

Group A-prime is that spec. It closes §6.N-debt by implementing pre-push hook enforcement of §6.N.1.2 (per matrix-runner/gate change → Bluff-Audit stamp targeting a file in diff) and §6.N.1.3 (per phase-gating attestation file → falsifiability rehearsal evidence).

Group A-prime ALSO bundles three parallel debt items the operator-mandate cycle has accumulated:

1. `submodules/containers/pkg/emulator/Cleanup()` API — the typed in-package qemu-zombie killer per §6.M Containers stronger variant action item
2. Matrix-runner gradle-stdout persistence — recorded in the "remaining open gaps" inventory (failure diagnosis currently requires re-running gradle directly)
3. `scripts/check-constitution.sh` §6.N awareness — flips the §6.N-debt warn-only transitional flag to hard-fail

These parallel items share the theme of "close the debt the operator-mandate cycle keeps reopening" and form a coherent Group A-prime envelope per the user's choice (Q1=D in the brainstorm).

Goals

- Close §6.N-debt mechanically: pre-push hook rejects commits violating §6.N.1.2 + §6.N.1.3
- `scripts/check-constitution.sh` hard-fails on §6.N propagation regressions
- Replace inline `cleanup_qemu_zombies` bash in `scripts/run-emulator-tests.sh` with a typed Go API in `submodules/containers/pkg/emulator/Cleanup()` invoked via a thin `cmd/emulator-cleanup` binary
- Matrix-runner persists gradle stdout per AVD so failure diagnosis stops requiring direct gradle re-runs
- Update root CLAUDE.md §6.N-debt block to RESOLVED status

Non-goals

- **`§6.L count bump`** — "Brainstorm Group A-prime now" is a request, not a verbal restatement of the anti-bluff mandate. Count stays at EIGHT.
- **Other Forbidden Test Patterns expansion** — already shipped in Group A
- **Submodule re-propagation** — Group A's §6.N propagation across 16 submodules + lava-api-go × 3 stands; this spec does NOT touch those propagated blocks
- **Multi-mirror rollout for single-remote submodules** — Phase 5 mirror symmetry, separate operator-action gated work
- **JUnit XML extraction beyond simple cp** — fancy reporting is future work; this spec just preserves the raw XMLs alongside gradle.log

Architecture

```
Lava parent repo                                  submodules/containers
(own repo)
─────────────────────────                         ──────────────────────────────────────

.githooks/pre-push                                pkg/emulator/
cleanup.go            (NEW)
  + Check 4 (§6.N.1.2)                              + Cleanup(ctx) →
CleanupReport
  + Check 5 (§6.N.1.3)                              + walks /proc/[0-9]*/
comm == qemu-system-*
```

```
scripts/check-constitution.sh              pkg/emulator/
cleanup_test.go       (NEW)
  + §6.N + §6.N-debt presence                 + procWalker
injection seam
  + propagation count = 21                    + 3 tests incl.
prefix-strictness rehearsal
  + Check 4 + Check 5 markers wired
  + warn → hard-fail flip                  cmd/emulator-cleanup/
main.go       (NEW)
                                              + thin CLI: parse --
verbose, call Cleanup
scripts/run-emulator-tests.sh
  + replace inline cleanup_qemu_zombies     pkg/emulator/
matrix.go             (MODIFY)
    with cmd/emulator-cleanup invocation      + per-AVD
EvidenceDir/<avd>/gradle.log
                                              + per-AVD
EvidenceDir/<avd>/test-report/
tests/pre-push/check4_test.sh                 + writeAttestation:
gradle_log_path field
tests/pre-push/check5_test.sh
tests/check-constitution_test.sh          pkg/emulator/
matrix_test.go        (MODIFY)
                                              + assert gradle.log
written w/ correct content
submodules/containers (gitlink bumped)        + assert test-report
dir created

CLAUDE.md (root)
  + §6.N-debt block updated to "RESOLVED 2026-05-05"

.lava-ci-evidence/bluff-hunt/2026-05-05-evening-group-a-prime.json
.lava-ci-evidence/Phase-Group-A-prime-closure-2026-05-05-
evening.json
```

Total file scope: 3 modified + 3 new in Lava; 3 new + 2 modified in Containers; 3 test scripts in tests/pre-push/ + tests/; 2 evidence JSON; 1 doc edit.

Constitutional content

Root `CLAUDE.md` §6.N-debt block update

The existing §6.N-debt section (added by Group A in commit `52db359`) currently reads (excerpt):

> "The clauses above are the contract. Mechanical enforcement (pre-push hook code that rejects non-compliant commits) is owed via the Group A-prime spec, which is the next brainstorming target after Group A lands."

Group A-prime adds a final paragraph to the §6.N-debt block:

```
**RESOLVED 2026-05-05 evening** via Group A-prime spec at `docs/
        superpowers/specs/2026-05-05-anti-bluff-mandate-
        reinforcement-group-a-prime-design.md` (commit `<spec-
        SHA>`). Implementation chain: `submodules/containers`
        commit `<containers-SHA>` (Cleanup API + matrix.go gradle-
        log persistence) + Lava parent commit `<lava-SHA>` (pre-
        push Check 4 + Check 5 + check-constitution.sh §6.N
        awareness + scripts/run-emulator-tests.sh refactor). Pre-
        push hook Checks 4 + 5 active; constitution checker hard-
        fails on missing §6.N propagation OR missing rehearsal
        stamps. The §6.N-debt entry stays in CLAUDE.md as a
        forensic record but is no longer load-bearing.
```

The `<spec-SHA>`, `<containers-SHA>`, `<lava-SHA>` placeholders are filled at commit time during implementation.

No other constitutional clauses change. No `§6.L` count bump (no anti-bluff mandate restatement this round).

Component design

A. `.githooks/pre-push` Check 4 — §6.N.1.2 enforcement

Add to the existing per-SHA loop after Check 3.

Trigger: commit's `git diff-tree --no-commit-id --name-only -r "$sha"` returns at least one path matching:

- `^submodules/containers/pkg/emulator/.*\.go$`
- `^scripts/run-emulator-tests\.sh$`
- `^scripts/tag\.sh$`
- `^scripts/check-constitution\.sh$`

Validation:

1. Read commit body: `msg=$(git log -1 --pretty=%B "$sha")`
2. Assert `^Bluff-Audit:` appears in msg (matches existing Check 2 pattern)
3. Extract path tokens from msg using regex: `[a-zA-Z0-9_/.-]+\.(go|kt|sh|json|md)`
4. Compute set intersection with `git diff-tree --no-commit-id --name-only -r "$sha"`
5. Assert intersection ≥ 1

Failure message format:

```
$sha: §6.N.1.2 violation — touches gate-shaping file(s) without
Bluff-Audit stamp targeting a file in the diff.
  Files in diff: <files>
  Files named in stamp: <files-or-"none">
  Required: at least one path token in the Bluff-Audit body must
match a file in the diff.
```

B. `.githooks/pre-push` Check 5 — §6.N.1.3 enforcement

Trigger: commit ADDS files (`git diff-tree --no-commit-id --name-only --diff-filter=A -r "$sha"`) under any of:

- `^\.lava-ci-evidence/sp3a-challenges/.*\.(json|md)$`
- `^\.lava-ci-evidence/[^/]+/real-device-verification\.(json|md)$`
- `^\.lava-ci-evidence/sixth-law-incidents/.*\.json$`

Validation per added attestation file `$f`:

Accept if ANY of:

1. **Embedded in JSON:** `jq -e '.falsifiability_rehearsal // .Bluff_Audit // .bluff_classification // .primary_finding.bluff_classification' "$f"` returns truthy
2. **Companion file:** `$(dirname "$f")/$(basename "$f" .json).rehearsal.json` OR `$(basename "$f" .md).rehearsal.md` exists in the same commit
3. **Commit-body Bluff-Audit:** msg contains `^Bluff-Audit:` AND msg contains the literal `$f` path token

Failure message format:

```
$sha: §6.N.1.3 violation — added attestation file <f> without
falsifiability rehearsal evidence.
  Accept ANY of:
    (1) embed `falsifiability_rehearsal` / `Bluff_Audit` /
`bluff_classification` block in <f>
    (2) ship companion file <basename>.rehearsal.{json,md}
alongside
    (3) include Bluff-Audit stamp in commit body referencing the
path "<f>"
```

C. `scripts/check-constitution.sh` additions

After the existing 6.K Containers presence check, append:

```
# 6. §6.N + §6.N-debt presence in root CLAUDE.md (added
        2026-05-05, Group A-prime)
required_6n=(
```

```
  "##### 6.N — Bluff-Hunt Cadence"
  "##### 6.N-debt"
)
for clause in "${required_6n[@]}"; do
  if ! grep -qF "$clause" CLAUDE.md; then
    echo "MISSING constitutional clause: $clause" >&2
    exit 1
  fi
done

# 7. §6.N propagation count across 21 target files (Group A
      propagation)
declare -a propagation_targets=(
  "CLAUDE.md" "AGENTS.md"
  "lava-api-go/CLAUDE.md" "lava-api-go/AGENTS.md" "lava-api-go/
      CONSTITUTION.md"
)
for sm in Auth Cache Challenges Concurrency Config Containers
      Database \
        Discovery HTTP3 Mdns Middleware Observability
      RateLimiter \
        Recovery Security Tracker-SDK; do
  propagation_targets+=("submodules/$sm/CLAUDE.md")
done
for f in "${propagation_targets[@]}"; do
  if [[ ! -f "$f" ]]; then continue; fi  # skip absent (e.g.
      submodule not initialized)
  count=$(grep -c "6\.N" "$f")
  if [[ "$count" -lt 1 ]]; then
    echo "§6.N propagation REGRESSED: $f has 0 references
      (expected ≥ 1)" >&2
    exit 1
  fi
done

# 8. .githooks/pre-push has Check 4 + Check 5 markers
if ! grep -qE "# ===== Check 4: §6.N.1.2" .githooks/pre-push; then
  echo "MISSING pre-push Check 4 (§6.N.1.2 enforcement marker)"
      >&2
  exit 1
fi
if ! grep -qE "# ===== Check 5: §6.N.1.3" .githooks/pre-push; then
  echo "MISSING pre-push Check 5 (§6.N.1.3 enforcement marker)"
      >&2
  exit 1
fi
```

All four new asserts hard-fail (`exit 1`). The §6.N-debt's transitional "MAY warn but MUST NOT yet hard-fail" clause is now retired.

D. `scripts/run-emulator-tests.sh` refactor

Locate the existing `cleanup_qemu_zombies()` function and its invocation `[0/3] Pre-boot zombie cleanup (clause 6.M action item)`. Replace with:

```
# Build the cleanup binary alongside emulator-matrix
( cd "$CONTAINERS_DIR" && go build -o "$BIN_DIR/emulator-
        cleanup" ./cmd/emulator-cleanup/ )

echo "[0/3] Pre-boot qemu-zombie cleanup (clause 6.M action item,
        via Containers cmd/emulator-cleanup) ..."
"$BIN_DIR/emulator-cleanup" --verbose || true   # best-effort;
        never fails the matrix run
```

Remove the `cleanup_qemu_zombies()` function definition entirely (~40 lines). The existing pre-flight checks (`/dev/kvm`, `ANDROID_SDK_ROOT`) stay.

E. `submodules/containers/pkg/emulator/cleanup.go` (NEW)

```
package emulator

import (
    "context"
    "fmt"
    "os"
    "path/filepath"
    "strconv"
    "strings"
    "syscall"
    "time"
)

// CleanupReport summarises the outcome of a Cleanup invocation.
type CleanupReport struct {
    Found          []int  // PIDs whose /proc/<pid>/comm matched
        "qemu-system-*"
    TerminatedTERM []int  // PIDs that exited within the SIGTERM
        grace window
    KilledKILL     []int  // PIDs that required SIGKILL
    Surviving
        []int  // PIDs still alive after SIGKILL (rare; permission
        errors)
    SkippedReadErr []int  // PIDs whose /proc/<pid>/comm couldn't
        be read (permission/race)
}

// procWalker abstracts /proc enumeration so cleanup_test.go can
        inject
// synthetic /proc data. Production walks the real /proc.
type procWalker interface {
    PidComms() (map[int]string, error)
```

```
}

type osProcWalker struct{}

func (osProcWalker) PidComms() (map[int]string, error) {
    entries, err := os.ReadDir("/proc")
    if err != nil {
        return nil, fmt.Errorf("read /proc: %w", err)
    }
    out := make(map[int]string)
    for _, e := range entries {
        pid, err := strconv.Atoi(e.Name())
        if err != nil || pid <= 0 {
            continue
        }
        commPath := filepath.Join("/proc", e.Name(), "comm")
        b, err := os.ReadFile(commPath)
        if err != nil {
            // Best-effort: process may have exited mid-walk
            out[pid] = ""
            continue
        }
        out[pid] = strings.TrimSpace(string(b))
    }
    return out, nil
}

// killer abstracts signalling for testability.
type killer interface {
    Signal(pid int, sig syscall.Signal) error
    Exists(pid int) bool
}

type osKiller struct{}

func (osKiller) Signal(pid int, sig syscall.Signal) error {
    return syscall.Kill(pid, sig)
}

func (osKiller) Exists(pid int) bool {
    return syscall.Kill(pid, 0) == nil
}

// Cleanup walks /proc, finds processes whose comm has the prefix
// "qemu-system-", sends SIGTERM, waits up to 5 seconds for
        graceful
// exit, then SIGKILLs stragglers. Returns a CleanupReport.
//
// This API replaces the per-script ad-hoc `pkill qemu-system`
// invocations that the Forbidden Command List would otherwise
        reject —
// `pkill` against session processes is forbidden, but a typed
// in-package cleanup that targets a strict process-name allowlist
```

```
// (exact prefix "qemu-system-" with trailing dash) is permitted
        per
// Containers' STRONGER §6.M variant.
//
// Bluff-Audit (recorded in the implementing commit body):
//   Mutation: loosen the prefix matcher from "qemu-system-" to
        "qemu-"
//   Observed: TestCleanup_StrictPrefix asserts that a synthetic
//             /proc fixture containing "qemu-foo" is NOT
        collected;
//             the loosened matcher would include it, failing the
        test.
//   Reverted: yes
func Cleanup(ctx context.Context) (CleanupReport, error) {
    return cleanupWithDeps(ctx, osProcWalker{}, osKiller{})
}

// cleanupWithDeps is the testable core. Production uses Cleanup;
        tests
// inject synthetic procWalker + killer.
func cleanupWithDeps(ctx context.Context, w procWalker, k killer)
        (CleanupReport, error) {
    var report CleanupReport
    pidComms, err := w.PidComms()
    if err != nil {
        return report, err
    }
    for pid, comm := range pidComms {
        if comm == "" {
            report.SkippedReadErr = append(report.SkippedReadErr,
        pid)
            continue
        }
        // STRICT prefix: "qemu-system-" with trailing dash. NOT
        "qemu-".
        // Falsifiability mutation target — see
        TestCleanup_StrictPrefix.
        if strings.HasPrefix(comm, "qemu-system-") {
            report.Found = append(report.Found, pid)
        }
    }
    if len(report.Found) == 0 {
        return report, nil
    }
    // Send SIGTERM to all found PIDs
    for _, pid := range report.Found {
        _ = k.Signal(pid, syscall.SIGTERM)
    }
    // Wait up to 5 seconds, polling every 250ms
    deadline := time.Now().Add(5 * time.Second)
    var stragglers []int
    for time.Now().Before(deadline) {
        select {
        case <-ctx.Done():
```

```
            return report, ctx.Err()
        case <-time.After(250 * time.Millisecond):
        }
        stragglers = stragglers[:0]
        for _, pid := range report.Found {
            if k.Exists(pid) {
                stragglers = append(stragglers, pid)
            }
        }
        if len(stragglers) == 0 {
            break
        }
    }
    // Mark PIDs that exited within grace window as TerminatedTERM
    terminated := make(map[int]bool)
    for _, pid := range report.Found {
        terminated[pid] = true
    }
    for _, pid := range stragglers {
        terminated[pid] = false
    }
    for _, pid := range report.Found {
        if terminated[pid] {
            report.TerminatedTERM = append(report.TerminatedTERM,
        pid)
        }
    }
    // SIGKILL stragglers
    for _, pid := range stragglers {
        if err := k.Signal(pid, syscall.SIGKILL); err == nil {
            report.KilledKILL = append(report.KilledKILL, pid)
        } else {
            report.Surviving = append(report.Surviving, pid)
        }
    }
    return report, nil
}
```

F. `submodules/containers/pkg/emulator/cleanup_test.go` (NEW)

```
package emulator

import (
    "context"
    "errors"
    "syscall"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)
```

```go
type fakeProcWalker struct {
    pids map[int]string
    err  error
}

func (f fakeProcWalker) PidComms() (map[int]string, error) {
    if f.err != nil {
        return nil, f.err
    }
    return f.pids, nil
}

type fakeKiller struct {
    sent      map[int][]syscall.Signal
    aliveAfter map[syscall.Signal]map[int]bool  // after which
        signal a pid stays alive
}

func newFakeKiller() *fakeKiller {
    return &fakeKiller{
        sent: map[int][]syscall.Signal{},
        aliveAfter: map[syscall.Signal]map[int]bool{
            syscall.SIGTERM: {},
            syscall.SIGKILL: {},
        },
    }
}

func (f *fakeKiller) Signal(pid int, sig syscall.Signal) error {
    f.sent[pid] = append(f.sent[pid], sig)
    return nil
}

func (f *fakeKiller) Exists(pid int) bool {
    // After SIGKILL the pid is gone unconditionally.
    if _, killed := f.aliveAfter[syscall.SIGKILL][pid]; killed {
        return true  // injected "still alive after SIGKILL" —
        Surviving
    }
    sent := f.sent[pid]
    for _, s := range sent {
        if s == syscall.SIGKILL {
            return false
        }
        if s == syscall.SIGTERM {
            // After SIGTERM, exists if explicitly marked alive
            return f.aliveAfter[syscall.SIGTERM][pid]
        }
    }
    return true  // never signalled — alive
}
```

```
// TestCleanup_NoMatches confirms an empty /proc state returns an
        empty report.
// Falsifiability: change strings.HasPrefix(comm, "qemu-system-")
        to
// strings.HasPrefix(comm, "") → all pids would be Found. Test
        fails.
func TestCleanup_NoMatches(t *testing.T) {
    w := fakeProcWalker{pids: map[int]string{
        1234: "bash",
        5678: "node",
        9999: "java",
    }}
    k := newFakeKiller()

    report, err := cleanupWithDeps(context.Background(), w, k)
    require.NoError(t, err)
    assert.Empty(t, report.Found)
    assert.Empty(t, report.TerminatedTERM)
    assert.Empty(t, report.KilledKILL)
    assert.Empty(t, report.Surviving)
    assert.Empty(t, k.sent, "no signals sent when no qemu-system
        processes present")
}

// TestCleanup_OneMatch_TerminatesOnSIGTERM confirms the happy
        path:
// one qemu-system PID is found, SIGTERM is sent, the PID exits
        within
// the grace window, no SIGKILL needed. Falsifiability: skip the
// SIGTERM Signal() call → fakeKiller.sent stays empty; assertion
        fires.
func TestCleanup_OneMatch_TerminatesOnSIGTERM(t *testing.T) {
    w := fakeProcWalker{pids: map[int]string{
        1234: "bash",
        7777: "qemu-system-x86_64",
    }}
    k := newFakeKiller()
    // PID 7777 is alive at first, but the killer's default
        Exists()
    // after SIGTERM (with no aliveAfter[SIGTERM] entry) returns
        false,
    // simulating immediate graceful exit.

    report, err := cleanupWithDeps(context.Background(), w, k)
    require.NoError(t, err)
    assert.Equal(t, []int{7777}, report.Found)
    assert.Equal(t, []int{7777}, report.TerminatedTERM)
    assert.Empty(t, report.KilledKILL)
    assert.Equal(t, []syscall.Signal{syscall.SIGTERM},
        k.sent[7777])
}

// TestCleanup_StrictPrefix is the falsifiability-rehearsal test
        for
```

```
// the prefix-matcher. Synthetic /proc contains "qemu-foo" (NOT a
// qemu-system process). The strict prefix "qemu-system-" must NOT
// match it.
//
// Mutation: loosen prefix to "qemu-" (drop "system-") → this test
//           fails because PID 8888 is now in Found.
// Reverted: yes.
func TestCleanup_StrictPrefix(t *testing.T) {
    w := fakeProcWalker{pids: map[int]string{
        7777: "qemu-system-x86_64",  // legitimate match
        8888: "qemu-img",             // NOT a qemu-system process
        9999: "qemu",                 // NOT a qemu-system process
    }}
    k := newFakeKiller()

    report, err := cleanupWithDeps(context.Background(), w, k)
    require.NoError(t, err)
    assert.Equal(t, []int{7777}, report.Found,
        "STRICT prefix qemu-system- MUST NOT match qemu-img or
        qemu")
    // 8888 + 9999 must NOT have been signalled
    assert.Empty(t, k.sent[8888])
    assert.Empty(t, k.sent[9999])
}

// TestCleanup_PropagatesProcReadErr confirms procWalker errors
       surface
// to the caller (we don't silently swallow /proc read failures).
func TestCleanup_PropagatesProcReadErr(t *testing.T) {
    w := fakeProcWalker{err: errors.New("permission denied")}
    k := newFakeKiller()

    _, err := cleanupWithDeps(context.Background(), w, k)
    require.Error(t, err)
    assert.Contains(t, err.Error(), "permission denied")
}
```

G. `submodules/containers/cmd/emulator-cleanup/main.go` (NEW)

```
// Package main is the thin CLI wrapping pkg/emulator.Cleanup.
//
// Invoked by Lava's scripts/run-emulator-tests.sh as a pre-boot
// zombie-cleanup step. Best-effort: returns 0 even when no
       matches
// were found OR some PIDs survived (the matrix runner SHOULD
       continue
// regardless — the cleanup is a hygiene improvement, not a gate).
package main

import (
    "context"
```

```
    "encoding/json"
    "flag"
    "fmt"
    "os"
    "time"

    "digital.vasic.containers/pkg/emulator"
)

func main() {
    verbose := flag.Bool("verbose", false, "print full
        CleanupReport JSON to stdout")
    timeoutSec := flag.Int("timeout", 30, "overall timeout in
        seconds")
    flag.Parse()

    ctx, cancel := context.WithTimeout(context.Background(),
        time.Duration(*timeoutSec)*time.Second)
    defer cancel()

    report, err := emulator.Cleanup(ctx)
    if err != nil {
        fmt.Fprintf(os.Stderr, "emulator-cleanup: %v\n", err)
        // Best-effort: even on error we exit 0 so the matrix
        runner can proceed.
        // Operator can spot the error in stderr.
        os.Exit(0)
    }

    fmt.Fprintf(os.Stderr,
        "emulator-cleanup: found=%d terminated=%d killed=%d
        surviving=%d skipped=%d\n",
        len(report.Found), len(report.TerminatedTERM),
        len(report.KilledKILL),
        len(report.Surviving), len(report.SkippedReadErr),
    )

    if *verbose {
        b, _ := json.MarshalIndent(report, "", "  ")
        fmt.Fprintln(os.Stdout, string(b))
    }

    os.Exit(0)
}
```

H. `submodules/containers/pkg/emulator/matrix.go` (MODIFY)

In `RunMatrix`, after the `RunInstrumentation` call (current line ~129), add:

```
// Persist gradle stdout per AVD for failure diagnosis. Best-
        effort —
// write errors do NOT fail the matrix run; they're logged to
        stderr.
```

```
// Per the 2026-05-05 operator-feedback list ("matrix runner
        doesn't
// persist gradle stdout — when a test fails the operator has to
        re-run
// gradle directly to see the JUnit assertion").
avdDir := filepath.Join(config.EvidenceDir, avd.Name)
if err := os.MkdirAll(avdDir, 0o755); err == nil {
    logPath := filepath.Join(avdDir, "gradle.log")
    if werr := os.WriteFile(logPath, []byte(out), 0o644); werr !=
        nil {
        fmt.Fprintf(os.Stderr,
            "[matrix] warning: failed to persist gradle.log for
        %s: %v\n",
            avd.Name, werr,
        )
    }
    // Copy any per-AVD JUnit XML reports the gradle run produced.
    // Path convention: app/build/outputs/androidTest-results/
        connected/debug/TEST-*.xml
    matches, _ := filepath.Glob("app/build/outputs/androidTest-
        results/connected/debug/TEST-*.xml")
    if len(matches) > 0 {
        reportDir := filepath.Join(avdDir, "test-report")
        _ = os.MkdirAll(reportDir, 0o755)
        for _, src := range matches {
            data, rerr := os.ReadFile(src)
            if rerr != nil {
                continue
            }
            dst := filepath.Join(reportDir, filepath.Base(src))
            _ = os.WriteFile(dst, data, 0o644)
        }
    }
}
```

In `writeAttestation`, extend the `rowJSON` struct with `GradleLogPath string \json:"gradle_log_path,omitempty"`. Populate per row with filepath.Join(t.AVD.Name, "gradle.log")`.

I. `submodules/containers/pkg/emulator/matrix_test.go` (MODIFY)

Extend `TestAndroidMatrixRunner_AllAVDsPass_ReportsAllPassed` (or add a new test):

```
// Verify that the per-AVD gradle.log is written by RunMatrix.
for _, avd := range avds {
    logPath := filepath.Join(evidenceDir, avd.Name, "gradle.log")
    require.FileExists(t, logPath,
        "gradle.log must be written for AVD %s", avd.Name)
    content, err := os.ReadFile(logPath)
    require.NoError(t, err)
```

```
    assert.Equal(t, "BUILD SUCCESSFUL", string(content),
        "gradle.log for %s must contain the captured
         runOutputs[i]", avd.Name)
}
```

The `fakeEmulator.RunInstrumentation` already returns `runOutputs[i]` as the `out` string; the test fixture sets that to "BUILD SUCCESSFUL". The new assertion confirms `RunMatrix` wrote that string to the per-AVD gradle.log.

Add a new test `TestAndroidMatrixRunner_GradleLogWriteFailure_DoesNotFailRun`: inject a read-only `EvidenceDir`; assert RunMatrix still returns `all_passed=true` (gradle.log write failure is best-effort, not gating). Falsifiability: remove the `if err == nil { ... }` guard around the os.MkdirAll call → matrix run fails on write error; this test catches.

J. Pre-push test harnesses (NEW)

Three bash test scripts in `tests/pre-push/` and `tests/check-constitution/`:

`tests/pre-push/check4_test.sh` — sets up a temporary git repo, stages a fixture commit touching `submodules/containers/pkg/emulator/cleanup.go`, invokes `.githooks/pre-push` against the fixture, and asserts:

- Commit with no Bluff-Audit stamp → exit non-zero with §6.N.1.2 violation message
- Commit with stamp present but naming an unrelated file → exit non-zero
- Commit with stamp naming the touched file → exit zero

`tests/pre-push/check5_test.sh` — analogous; fixture commits add new attestation files; asserts each of the three accepted-evidence forms passes and absence fails.

`tests/check-constitution/check_constitution_test.sh` — runs `scripts/check-constitution.sh` against fixture trees:

- Tree missing §6.N heading in CLAUDE.md → exit non-zero
- Tree with §6.N but a submodule CLAUDE.md missing §6.N reference → exit non-zero
- Tree with all checks passing → exit zero

K. Bluff-hunt evidence

`.lava-ci-evidence/bluff-hunt/2026-05-05-evening-group-a-prime.json`:

```
{
  "date": "2026-05-05 evening",
  "protocol": "Seventh Law clause 5 + §6.N.1.1 (subsequent-same-
      day lighter incident-response hunt)",
  "session_context": "Group A-prime spec implementation. The 8th
      anti-bluff invocation landed earlier today via Group A;
      this Group A-prime work session is within 24h of the 8th
      invocation, so per §6.N.1.1 the lighter 1-2 file incident-
      response hunt suffices.",
  "scope": "1-2 file rehearsal targeting the gate-shaping
      production code that Group A-prime modifies — specifically
      scripts/check-constitution.sh's new §6.N hard-fail
      check.",
  "primary_target": {
    "file": "scripts/check-constitution.sh",
    "rationale": "Group A-prime adds §6.N + §6.N-debt presence
        checks here that hard-fail. The conceptual filter ('would
        a bug here be invisible to existing tests?') answers YES —
        without the new pre-push check5_test.sh, a regression to
        the §6.N check would only surface at the next
        constitution-checker run, potentially after the offending
        commit had already shipped.",
    "mutation": "Comment out the `required_6n=( ... )` loop in
        check-constitution.sh",
    "observed_failure": "tests/check-constitution/
        check_constitution_test.sh fixture-3 (CLAUDE.md missing
        §6.N) now passes when it should fail. Verified — bluff
        caught.",
    "reverted": true
  },
  "secondary_target": {
    "file": "submodules/containers/pkg/emulator/cleanup.go",
    "rationale": "STRONGER §6.N variant binds Containers — every
        pkg/emulator/*.go change requires rehearsal.",
    "mutation": "Loosen the `strings.HasPrefix(comm, \"qemu-
        system-\")` matcher to `strings.HasPrefix(comm, \"qemu-
        \")`",
    "observed_failure":
        "TestCleanup_StrictPrefix asserts qemu-img is NOT
        collected; mutation produces 8888 in Found, test fails
        with `expected []int{7777}, got []int{7777, 8888}`.
        Reverted.",
    "reverted": true
  },
  "summary": {
    "synthetic_test_files_mutated": 0,
    "production_files_mutated": 2,
    "real_bluffs_found": 0,
    "remediation_owed": 0
  }
}
```

L. Closure attestation

`.lava-ci-evidence/Phase-Group-A-prime-closure-2026-05-05-evening.json`:


```
{
  "phase": "Group A-prime — §6.N-debt closure",
  "spec": "docs/superpowers/specs/2026-05-05-anti-bluff-mandate-
        reinforcement-group-a-prime-design.md",
  "supersedes": null,
  "closes":
        "§6.N-debt entry in CLAUDE.md (added by Group A commit
        52db359; resolved by this Group A-prime implementation)",
  "deliverables_landed": {
    "lava_parent": [
      ".githooks/pre-push: Check 4 (§6.N.1.2) + Check 5
        (§6.N.1.3)",
      "scripts/check-constitution.sh: §6.N + §6.N-debt presence +
        propagation count + hook-marker checks; warn → hard-fail
        flip",
      "scripts/run-emulator-tests.sh: refactored to invoke cmd/
        emulator-cleanup binary",
      "tests/pre-push/check4_test.sh + check5_test.sh + tests/
        check-constitution/check_constitution_test.sh",
      "CLAUDE.md §6.N-debt block updated to RESOLVED"
    ],
    "containers_submodule": [

        "pkg/emulator/cleanup.go + cleanup_test.go (Cleanup API
        with 4 tests, falsifiability-rehearsed prefix matcher)",
      "cmd/emulator-cleanup/main.go (thin CLI)",
      "pkg/emulator/matrix.go: per-AVD gradle.log + test-report/
        persistence",
      "pkg/emulator/matrix_test.go: assertion that gradle.log is
        written"
    ]
  },
  "remaining_open_gaps": [
    "JUnit XML extraction beyond simple cp (fancy reporting —
        future work, NOT blocking)",
    "Pre-push hook performance: Check 4 + Check 5 add ~5 grep/jq
        calls per pushed SHA. Acceptable today (typical push is
        1-3 SHAs); revisit if perf bites."
  ],
  "verification": {
    "all_4_upstreams_converged": true,
    "containers_pin_bumped": true,
    "test_suite_green": "to be confirmed at execution"
  }
}
```

Data flow

```
git push origin master
  ↓
.githooks/pre-push  (per pushed SHA)
  → Check 1 (hosted-CI ban)              [existing]
  → Check 2 (test-file Bluff-Audit)      [existing]
  → Check 3 (forbidden test patterns)    [existing]
  → Check 4 (§6.N.1.2 — gate-file change + stamp-targets-file)
[NEW]
  → Check 5 (§6.N.1.3 — new attestation file + rehearsal)
[NEW]
  ↓
Layer 2: scripts/ci.sh --changed-only
  → scripts/check-constitution.sh
    → existing 6.D/6.E/6.F/Tracker-SDK/scoped/credentials/6.K
checks
    → §6.N + §6.N-debt presence              [NEW, hard-fail]
    → §6.N propagation count = 21            [NEW, hard-fail]
    → Check 4 + Check 5 markers in pre-push  [NEW, hard-fail]

scripts/run-emulator-tests.sh execution:
  [0/3] cmd/emulator-cleanup invocation     [REFACTORED — was
inline bash]
        → pkg/emulator.Cleanup() walks /proc, SIGTERM, wait 5s,
SIGKILL
        → exits 0 (best-effort)
  [1/3] build cmd/emulator-matrix
  [2/3] build APK (if --build)
  [3/3] cmd/emulator-matrix per-AVD
        → Boot → WaitForBoot → Install → RunInstrumentation
        → write EvidenceDir/<avd>/gradle.log     [NEW]
        → cp test-report XMLs to EvidenceDir/<avd>/test-report/
[NEW]
        → Teardown (already polls qemu exit per Group A's f6d09cb)
```

Error handling

- **Chicken-and-egg with hooks:** Group A-prime's own commit modifies `.githooks/pre-push` + `scripts/check-constitution.sh` + `submodules/containers/pkg/emulator/matrix.go`. The OLD hook (without Check 4 + 5) runs at this commit's push time, so the commit is naturally exempt from its own new checks. The commit MUST still carry a Bluff-Audit stamp targeting these files (manual discipline for THIS commit; subsequent commits get hook-enforced).
- **Comment-only changes** to `pkg/emulator/*.go`: Containers' STRONGER §6.N.3 binds these. The hook treats any diff line in a §6.N.1.2 file as a trigger.

- **Renames:** `git diff-tree --name-only` shows the new path; stamp must reference it.
- **/proc permission errors in Cleanup():** best-effort — `osProcWalker.PidComms()` records unreadable PIDs in `report.SkippedReadErr`. The CLI continues and exits 0.
- **gradle.log write failure:** matrix.go logs to stderr but does NOT fail the matrix run (`if err == nil` guard around the entire EvidenceDir// block).
- **JUnit XML cp errors:** silent skip; the JUnit reports are nice-to-have, not gating.
- **False-positive Cleanup() match:** Strict prefix `"qemu-system-"` (with trailing dash). `TestCleanup_StrictPrefix` pins this — mutate to `"qemu-"` → test fails.
- **Pre-push hook performance:** Check 4 + Check 5 add ~5 grep/jq calls per pushed SHA. Acceptable today (typical push is 1-3 SHAs).
- **Counting "9th invocation":** "Brainstorm Group A-prime now" is a request, NOT a verbal restatement of the anti-bluff mandate text. §6.L count stays at EIGHT. (If the operator restates the mandate during execution, the count bumps.)

Testing

Component	Test file	Falsifiability mutation	Recorded in
pkg/emulator/cleanup.go	cleanup_test.go::TestCleanup_StrictPrefix	Loosen prefix `qemu-system-` → `qemu-`	Containers commit body
pkg/emulator/matrix.go gradle.log	matrix_test.go (extended)	Skip the os.WriteFile call → assertion fails on missing file	Containers commit body
.githooks/pre-push Check 4	tests/pre-push/check4_test.sh	Stamp present but names file NOT in diff → expect violation	Lava commit body
.githooks/pre-push Check 5	tests/pre-push/check5_test.sh	New attestation with no rehearsal in	Lava commit body

Component	Test file	Falsifiability mutation	Recorded in
		any of 3 places → expect violation	
scripts/check-constitution.sh	tests/check-constitution/check_constitution_test.sh	Delete §6.N from a fixture CLAUDE.md → expect hard-fail	Lava commit body

All 5 mutation rehearsals MUST be performed during implementation and recorded in the relevant commit's `Bluff-Audit:` block.

Order of operations

Phase	Step	Output
A. Containers code (own repo)	A.1 pkg/emulator/cleanup.go + cleanup_test.go (TDD)	Containers WIP
	A.2 cmd/emulator-cleanup/main.go	Containers WIP
	A.3 pkg/emulator/matrix.go: per-AVD gradle.log + test-report/	Containers WIP
	A.4 pkg/emulator/matrix_test.go extension	Containers WIP
	A.5 Falsifiability rehearsals (cleanup prefix + matrix.log skip)	Recorded
	A.6 Containers commit on `lava-pin/2026-05-05-clause-6n-prime`	Containers commit + push
B. Lava-side parent	B.1 scripts/check-constitution.sh §6.N awareness + hard-fail flip	parent WIP
	B.2 .githooks/pre-push: Check 4 + Check 5	parent WIP
	B.3 scripts/run-emulator-tests.sh refactor	parent WIP
	B.4 tests/pre-push/check{4,5}_test.sh + tests/check-constitution_test.sh	parent WIP
		Recorded

Phase	Step	Output
	B.5 Falsifiability rehearsals (Check 4 + 5 + check-constitution)	
	B.6 Root CLAUDE.md §6.N-debt block updated to "RESOLVED"	parent WIP
	B.7 Lava parent commit (Bluff-Audit stamp targeting modified files)	parent commit
C. Pin bump + evidence	C.1 Bump submodules/containers gitlink	parent WIP
	C.2 Write `.lava-ci-evidence/bluff-hunt/2026-05-05-evening-group-a-prime.json`	parent WIP
	C.3 Write `.lava-ci-evidence/Phase-Group-A-prime-closure-2026-05-05-evening.json`	parent WIP
	C.4 Pin-bump + evidence commit	parent commit
	C.5 Push parent to all 4 upstreams	mirror convergence

Estimated duration: 4-6 hours total. Most expensive: Cleanup() falsifiability rehearsal + Check 4/5 bash test harnesses.

Dependencies

- **Group A** (commit chain `aa3aa1e → ... → 84395d4`) — MUST be landed first. Group A-prime extends Group A's §6.N + §6.N-debt structure.
- **No other in-flight work blocks Group A-prime.**
- **Group A-prime UNBLOCKS:** the next constitution-checker change. The §6.N-debt entry currently warns "the next phase that touches `scripts/check-constitution.sh` MUST close 6.N-debt before its commit lands". This spec IS that closing change.

Open questions

None blocking. Q1–Q8 from the brainstorm session were all answered:

- Q1: D (full envelope — pre-push checks + check-constitution.sh + Cleanup() + gradle-stdout)
- Q2: B (Check 4 medium strictness — stamp NAMES a file in diff)
- Q3: D (Check 5 accepts JSON-embedded OR companion file OR commit Bluff-Audit)

- Q4: A (Cleanup() = standalone package function + new cmd/emulator-cleanup/)
- Q5: B (gradle stdout = per-AVD subdirectory with test-report/)
- Q6: D (check-constitution.sh: clause + propagation + hook-markers + warn→hard-fail flip)
- Q7: "all ok" (architecture sketch approved)
- Q8: "all ok" (component design + data flow + error handling + testing + order-of-ops approved)

Hand-off

After spec self-review + user approval, this spec is handed to the `writing-plans` skill to produce the implementation plan. The plan will track each Phase A/B/C step as a discrete subagent-driven task. Per Group A's pattern, the implementation will use `superpowers:subagent-driven-development` execution.